

# Dart Programming

# Object Oriented Design

# Inheritance

- Inheritance is a sharing of behaviour between two classes. It allows you to define a class that extends the functionality of another class. The extend keyword is used for inheriting from parent class.
- Note: Whenever you use inheritance, it always create a ( is-a ) relation between the parent and child class like Student is a Person, Truck is a Vehicle, Cow is a Animal etc.
- Dart supports single inheritance, which means that a class can only inherit from a single class.
- Dart does not support multiple inheritance which means that a class cannot inherit from multiple classes.

```
class ParentClass {  
    // Parent class code  
}  
  
class ChildClass extends ParentClass {  
    // Child class code  
}
```

# Inheritance

- Terminology
- Parent Class: The class whose properties and methods are inherited by another class is called parent class. It is also known as base class or super class.
- Child Class: The class that inherits the properties and methods of another class is called child class. It is also known as derived class or sub class.

# Example

```
class Person {  
    // Properties  
    String? name;  
    int? age;  
  
    // Method  
    void display() {  
        print("Name: $name");  
        print("Age: $age");  
    }  
}
```

```
}  
// Here In student class, we are extending the  
// properties and methods of the Person class  
class Student extends Person {  
    // Fields  
    String? schoolName;  
    String? schoolAddress;  
  
    // Method  
    void displaySchoolInfo() {  
        print("School Name: $schoolName");  
        print("School Address: $schoolAddress");  
    }  
}
```

# Example

- `void main() {`
- `// Creating an object of the Student class`
- `var student = Student();`
- `student.name = "John";`
- `student.age = 20;`
- `student.schoolName = "ABC School";`
- `student.schoolAddress = "New York";`
- `student.display();`
- `student.displaySchoolInfo();`
- `}`

# Advantages Of Inheritance In Dart

- It promotes reusability of the code and reduces redundant code.
- It helps to design a program in a better way.
- It makes code simpler, cleaner and saves time and money on maintenance.
- It facilitates the creation of class libraries.
- It can be used to enforce standard interface to all children classes.



# Example

```
• class Car{  
•   String color;  
•   int year;  
•   void start(){  
•     print("Car started");  
•   }  
• }  
• class Toyota extends Car{  
•   String model;  
•   int price;  
void showDetails(){  
•   print("Model: $model");  
•   print("Price: $price");  
• }  
• }
```

```
void main(){  
  var toyota = Toyota();  
  toyota.color = "Red";  
  toyota.year = 2020;  
  toyota.model = "Camry";  
  toyota.price = 20000;  
  toyota.start();  
  toyota.showDetails();  
}
```



# Inheritance of constructor

- Inheritance of constructor in Dart is a process of inheriting the constructor of the parent class to the child class. It is a way of reusing the code of the parent class.
- In this example below, there is class named Laptop with a constructor. There is another class named MacBook which extends the Laptop class. The MacBook class has its own constructor.

```
class Laptop {  
  // Constructor  
  Laptop() {  
    print("Laptop constructor");  
  }  
}  
  
class MacBook extends Laptop {  
  // Constructor  
  MacBook() {  
    print("MacBook constructor");  
  }  
}  
  
void main() {  
  var macbook = MacBook();  
}
```

# Inheritance Of Constructor With Parameters

```
class Laptop {  
    // Constructor  
    Laptop(String name, String color) {  
        print("Laptop constructor");  
        print("Name: $name");  
        print("Color: $color");  
    }  
}  
  
class MacBook extends Laptop {  
    // Constructor  
    MacBook(String name, String color) : super(name, color) {  
        print("MacBook constructor");  
    }  
}  
  
void main() {  
    var macbook = MacBook("MacBook Pro", "Silver");  
}
```

# Example

```
class Person {
    String name;
    int age;

    // Constructor
    Person(this.name, this.age);
}

class Student extends Person {
    int rollNumber;

    // Constructor
    Student(String name, int age, this.rollNumber) : super(name, age);
}

void main() {
    var student = Student("John", 20, 1);
    print("Student name: ${student.name}");
    print("Student age: ${student.age}");
    print("Student roll number: ${student.rollNumber}");
}
```

# Inheritance Of Constructor With Named Parameters

```
class Laptop {  
    // Constructor  
    Laptop({String name, String color}) {  
        print("Laptop constructor");  
        print("Name: $name");  
        print("Color: $color");  
    }  
}  
  
class MacBook extends Laptop {  
    // Constructor  
    MacBook({String name, String color}) : super(name: name, color: color) {  
        print("MacBook constructor");  
    }  
}  
  
void main() {  
    var macbook = MacBook(name: "MacBook Pro", color: "Silver");  
}
```

- Super is used to refer to the parent class. It is used to call the parent class's properties and methods.

```
class Laptop {  
    // Method  
    void show() {  
        print("Laptop show method");  
    }  
}  
  
class MacBook extends Laptop {  
    void show() {  
        super.show(); // Calling the show method of the parent class  
        print("MacBook show method");  
    }  
}  
  
void main() {  
    // Creating an object of the MacBook class  
    MacBook macbook = MacBook();  
    macbook.show();  
}
```



```
class Employee {  
    // Constructor  
    Employee(String name, double salary) {  
        print("Employee constructor");  
        print("Name: $name");  
        print("Salary: $salary");  
    }  
}  
  
class Manager extends Employee {  
    // Constructor  
    Manager(String name, double salary) : super(name, salary) {  
        print("Manager constructor");  
    }  
}  
  
void main() {  
    Manager manager = Manager("John", 25000.0);  
}
```

# Super With Multilevel Inheritance In Dart

```
class Laptop {  
  // Method  
  void display() {  
    print("Laptop display");  
  }  
}  
  
class MacBook extends Laptop {  
  // Method  
  void display() {  
    print("MacBook display");  
    super.display();  
  }  
}  
  
class MacBookPro extends MacBook {  
  // Method  
  void display() {  
    print("MacBookPro display");  
    super.display();  
  }  
}  
  
void main() {  
  var macbookpro = MacBookPro();  
  macbookpro.display();  
}
```



- Poly means many and morph means forms.
- Polymorphism is the ability of an object to take on many forms.
- As humans, we have the ability to take on many forms. We can be a student, a teacher, a parent, a friend, and so on.
- Similarly, in object-oriented programming, polymorphism is the ability of an object to take on many forms.
- Note: In the real world, polymorphism is updating or modifying the feature, function, or implementation that already exists in the parent class.

# Polymorphism By Method Overriding

- Method overriding is a technique in which you can create a method in the child class that has the same name as the method in the parent class.
- The method in the child class overrides the method in the parent class.

```
class ParentClass{  
    void functionName(){  
    }  
}  
class ChildClass extends ParentClass{  
    @override  
    void functionName(){  
    }  
}
```

- Note: If you don't write @override, the program still runs. But, it is a good practice to write @override.

```
class Animal {  
    void eat() {  
        print("Animal is eating");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void eat() {  
        print("Dog is eating");  
    }  
}  
  
void main() {  
    Animal animal = Animal();  
    animal.eat();  
  
    Dog dog = Dog();  
    dog.eat();  
}
```

```
class Car{
    void power(){
        print("It runs on petrol.");
    }
}

class Honda extends Car{
}

class Tesla extends Car{
    @Override
    void power(){
        print("It runs on electricity.");
    }
}

void main(){
    Honda honda=Honda();
    Tesla tesla=Tesla();

    honda.power();
    tesla.power();
}
```

```
class Employee{
    void salary(){
        print("Employee salary is \$1000.");
    }
}

class Manager extends Employee{
    @Override
    void salary(){
        print("Manager salary is \$2000.");
    }
}

class Developer extends Employee{
    @Override
    void salary(){
        print("Developer salary is \$3000.");
    }
}

void main(){
    Manager manager=Manager();
    Developer developer=Developer();

    manager.salary();
    developer.salary();
}
```

# Abstract Class

- Abstract classes are classes that cannot be initialized.
- It is used to define the behavior of a class that can be inherited by other classes. An abstract class is declared using the keyword `abstract`.

```
abstract class ClassName {  
    //Body of abstract class  
  
    method1();  
    method2();  
}
```

- An abstract method is a method that is declared without an implementation. It is declared with a semicolon (;) instead of a method body.
- Subclasses of an abstract class must implement all the abstract methods of the abstract class. It is used to achieve abstraction in the Dart programming language.



# Example

```
• abstract class Vehicle {  
•   // Abstract method  
•   void start();  
•   // Abstract method  
•   void stop();  
• }  
• class Car extends Vehicle {  
•   // Implementation of start()  
•   @override  
•   void start() {  
•       print('Car started');  
•   }  
•   // Implementation of stop()  
•   @override  
•   void stop() {  
•       print('Car stopped');  
•   }  
• }
```

```
• class Bike extends Vehicle {  
• // Implementation of start()  
• @override  
• void start() {  
•   print('Bike started');  
• }  
• // Implementation of stop()  
• @override  
• void stop() {  
•   print('Bike stopped');  
• }  
• }  
• void main() {  
•   Car car = Car();  
•   car.start();  
•   car.stop();  
•   Bike bike = Bike();  
•   bike.start();  
•   bike.stop();  
• }
```

# Abstract class constructor

- You can't create an object of an abstract class. However, you can define a constructor in an abstract class. The constructor of an abstract class is called when an object of a subclass is created.
- abstract class Bank {
  - String name;
  - double rate;
  - // Constructor
  - Bank(this.name, this.rate);
  - // Abstract method
  - void interest();
  - //Non-Abstract method: It have an implementation
  - void display() {
    - print('Bank Name: \$name');
  - }
- }

- class CIB extends Bank {
- // Constructor
- CIB(String name, double rate) : super(name, rate);
- // Implementation of interest()
- @override
- void interest() {
- print('The rate of interest of SBI is \$rate');
- }
- }
- class HSBC extends Bank {
- // Constructor
- HSBC(String name, double rate) : super(name, rate);
- // Implementation of interest()
- @override
- void interest() {
- print('The rate of interest of ICICI is \$rate');
- }
- }

- void main() {
- CIB cib = CIB('CIB', 8.4);
- HSBC hsbc = HSBC('ICICI', 7.3);
- cib.interest();
- hsbc.interest();
- hsbc.display();
- }

# Interfaces

- An interface defines a syntax that a class must follow.
- It is a contract that defines the capabilities of a class.
- It is used to achieve abstraction in the Dart programming language.
- When you implement an interface, you must implement all the properties and methods defined in the interface. Keyword `implements` is used to implement an interface.

```
class InterfaceName {  
    // code  
}  
  
class ClassName implements InterfaceName {  
    // code  
}
```



# Interface

- In dart there is no keyword interface but you can use class or abstract class to declare an interface.
- All classes implicitly define an interface. Mostly abstract class is used to declare an interface.

```
// creating an interface using abstract class
abstract class Person {
  canWalk();
  canRun();
}
```

- You must use the implements keyword to implement an interface. The class that implements an interface must implement all the methods and properties of the interface.

```
class Student implements Person {  
    // implementation of canWalk()  
    @override  
    canWalk() {  
        print('Student can walk');  
    }  
  
    // implementation of canRun()  
    @override  
    canRun() {  
        print('Student can run');  
    }  
}
```

```
// abstract class as interface
abstract class Vehicle {
    void start();
    void stop();
}

// implements interface
class Car implements Vehicle {
    @override
    void start() {
        print('Car started');
    }

    @override
    void stop() {
        print('Car stopped');
    }
}

void main() {
    var car = Car();
    car.start();
    car.stop();
}
```

# Multiple Interfaces

- `// abstract class as interface`
- abstract class Area {
  - `void area();`
  - `}`
- `// abstract class as interface`
- abstract class Perimeter {
  - `void perimeter();`
  - `}`

- `// implements multiple interfaces`
- `class Rectangle implements Area, Perimeter {`
- `// properties`
- `int length, breadth;`
- `// constructor`
- `Rectangle(this.length, this.breadth);`
- `// implementation of area()`
- `@Override`
- `void area() {`
- `print('The area of the rectangle is ${length * breadth}');`
- `}`
- `// implementation of perimeter()`
- `@Override`
- `void perimeter() {`
- `print('The perimeter of the rectangle is ${2 * (length + breadth)}');`
- `}`
- `}`

- void main() {
- Rectangle rectangle = Rectangle(10, 20);
- rectangle.area();
- rectangle.perimeter();
- }



# Difference Between Extends & Implements

| extends                                                                    | implements                                                                     |
|----------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Used to inherit a class in another class.                                  | Used to inherit a class as an interface in another class.                      |
| Gives complete method definition to sub-class.                             | Gives abstract method definition to sub-class.                                 |
| Only one class can be extended.                                            | Multiple classes can be implemented.                                           |
| It is optional to override the methods.                                    | Concrete class must override the methods of an interface.                      |
| Constructors of the superclass is called before the sub-class constructor. | Constructors of the superclass is not called before the sub-class constructor. |
| The super keyword is used to access the members of the superclass.         | Interface members can't be accessed using the super keyword.                   |
| Sub-class need not to override the fields of the superclass.               | Subclass must override the fields of the interface.                            |

# Example

- class A {
- void silly() {
- print("A's Silly");}}
- class B {
- void awesome() {
- print("B's Awesome");}}
- class C implements A, B {
- void silly() {
- print("C's Silly");}
- void awesome() {
- print("C's Awesome");}}
- class D extends B implements A {
- void silly() {
- print("D's Silly");}}

- `void main() {`
- `A c = new C();`
- `c.silly(); // prints C's Silly`
- `c.awesome(); // runs, but generates warning; prints C's Awesome`
- `D d = new D();`
- `d.awesome(); // prints B's Awesome`
- `d.silly(); // prints D's Silly}`
- Casting is a way of indicating to Dart that we want to treat an object of one type like an object of another type with a compatible interface.
- In the last example, `c.awesome()` generates a warning (visible in Dart Editor), because `c` is actually declared to be of type `A`, and `A` does not have an `awesome()` method.
- However, `c` is instantiated as a `C` object, which does have an `awesome()` method.

- `A c = new C();`
- There is a way to tell Dart that we want to treat `c` as a `C`, even after it was originally declared as an `A`.
- We can do it with a cast. In Dart, the `as` operator is used for casting. Using a cast, we can stop the call of `awesome()` from generating a warning.
- `(c as C).awesome(); // no warning generated; prints C's Awesome`
- We are specifying that `c` is to be thought of as a `C`, regardless of the fact that it was originally declared as an `A`. This is only safe because we know that `c` was initialized as a `C`.

- Mixins are a way of reusing the code in multiple classes.
- Mixins are declared using the keyword `mixin` followed by the mixin name.
- Three keywords are used while working with mixins: `mixin`, `with`, and `on`. It is possible to use multiple mixins in a class.
- Note: The `with` keyword is used to apply the mixin to the class. It promotes DRY(Don't Repeat Yourself) principle.

# Rules For Mixin

- Mixin can't be instantiated. You can't create object of mixin.
- Use the mixin to share the code between multiple classes.
- Mixin has no constructor and cannot be extended.
- It is possible to use multiple mixins in a class.



# Example

```
mixin CanFly {  
  void fly() {  
    print('I can fly');  
  }  
}  
  
mixin CanWalk {  
  void walk() {  
    print('I can walk');  
  }  
}  
  
class Bird with CanFly, CanWalk {  
  
}  
  
class Human with CanWalk {  
  
}  
  
void main() {  
  var bird = Bird();  
  bird.fly();  
  bird.walk();  
  
  var human = Human();  
  human.walk();  
}
```

- What Is Allowed For Mixin
  - You can add properties and static variables.
  - You can add regular, abstract, and static methods.
  - You can use one or more mixins in a class.
- What Is Not Allowed For Mixin
  - You can't define a constructor.
  - You can't extend a mixin.
  - You can't create an object of mixin.